Smart Text Intelligence System — Problem Statement and Project Plan

Oct 22, 2026 · @vv

Problem Statement

Computers cannot work with raw human language, so this project builds a pure-NLP system that turns text into numbers, applies classical algorithms, and extracts useful information from it.

> Human language is unstructured, and computers can only work with numbers. We need a system that converts raw text into numerical representations, applies classical NLP algorithms to it, and extracts useful information (a category, a sentiment, keywords, word similarity, or the likelihood of a sentence) without relying on LLMs or generative AI. We must also measure how well each approach works and where it fails.

The project has three goals:

- **Learn deeply:** understand the concept, the mathematics, and the failure cases of every technique, not just the library call.
- **Build practically:** produce a modular, tested Python codebase and a working application.
- **Be placement-ready:** end with a portfolio-quality GitHub repository that can be explained clearly in AI/ML interviews.

Scope

The project covers classical NLP only: everything from text preprocessing to statistical language models, and nothing generative.

In scope	Out of scope
Text preprocessing and tokenization	Retrieval-Augmented Generation (RAG)
Bag of Words, n-grams, TF-IDF	LLM APIs and LLM integration

In scope	Out of scope
Word2Vec, GloVe, FastText embeddings	Vector databases
Naive Bayes and Logistic Regression classifiers	Generative AI features
Statistical n-gram language models	
Text analysis and evaluation	
Streamlit interface, added last	

The restriction is deliberate: the aim is to show that the fundamentals are understood, not that an API can be called.

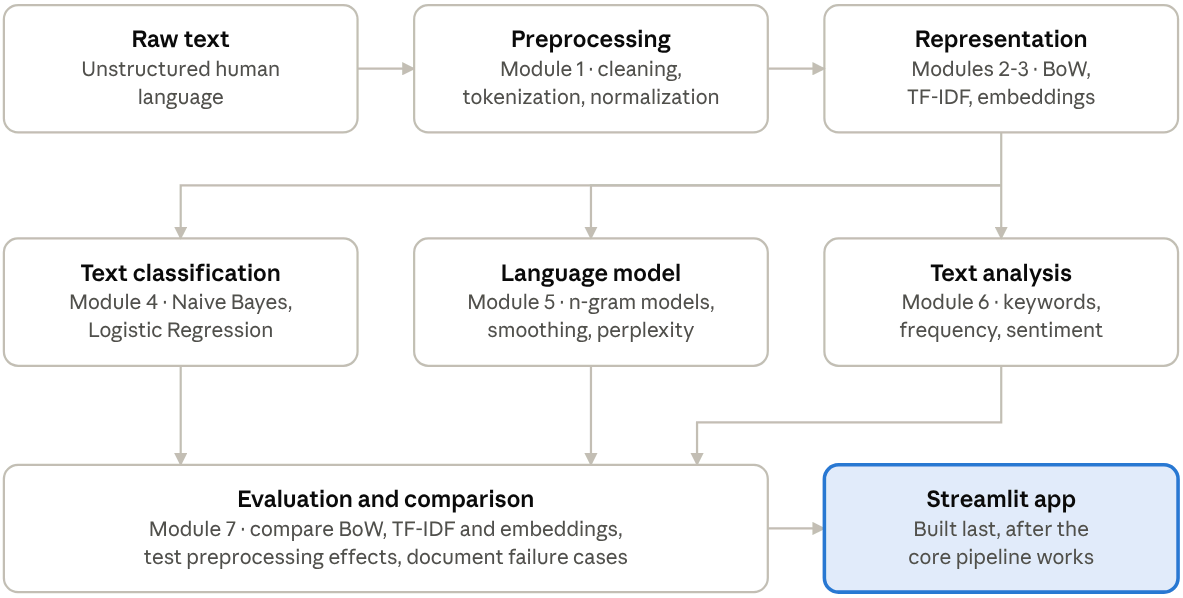
Approach

We work one module at a time, repeating the same seven-step loop in each, so every technique is understood before it is used.

1. **Concept:** what the technique is and why it exists.
2. **Math and algorithm:** the reasoning behind it, with worked numbers.
3. **Manual implementation:** a small from-scratch version where practical.
4. **Library version:** NLTK, spaCy, scikit-learn, or Gensim.
5. **Test on real text:** run both versions and compare their outputs.
6. **Evaluate:** measure with suitable metrics and examples.
7. **Document:** record advantages, limitations, and practical use cases.

The code stays modular and testable, so each module can be explained on its own in a technical interview. The modules also chain together into one growing pipeline, drawn below.

Every module feeds one pipeline that ends in the app



pipeline · 8 stages, 3 parallel branches

Preprocessed text becomes vectors, which feed classification, language modeling, and analysis; Module 7 compares the results before the app wraps them.

Module Plan

Seven modules, built in order, each producing code, tests, and a short write-up.

#	Module	Key topics	Built by hand	Library version
1	Text Preprocessing	Cleaning, tokenization, stop words, stemming, lemmatization, contractions, negation	Cleaner and tokenizer	NLTK, spaCy
2	Text Representation	One-hot, Bag of Words, n-grams, TF-IDF, sparsity	TF-IDF from the formula	CountVectorizer, TfidfVectorizer

#	Module	Key topics	Built by hand	Library version
3	Word Embeddings	Word2Vec (CBOW, Skip-gram), GloVe, FastText, similarity, OOV limits	Cosine similarity and vector arithmetic	Gensim
4	Text Classification	TF-IDF features, Naive Bayes, Logistic Regression, accuracy, precision, recall, F1	Naive Bayes classifier	scikit-learn
5	Statistical Language Modeling	Unigram, bigram, trigram models, MLE, Markov assumption, Laplace smoothing, perplexity	Bigram model with smoothing and perplexity	NLTK for verification
6	Text Analysis	Keyword extraction, word frequency, similarity, sentiment	Frequency and TF-IDF keyword ranking	NLTK, Gensim
7	Evaluation and Comparison	BoW vs TF-IDF vs embeddings, effect of preprocessing, failure cases	Evaluation harness	scikit-learn metrics

The TF-IDF weight that Module 2 implements by hand is:

```
\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log\frac{N}{\text{df}(t)}
```

Here tf(t, d) is how often term t appears in document d, N is the number of documents, and df(t) is how many documents contain t. Library versions add smoothing, so the by-hand output is checked against scikit-learn's formula rather than assumed to match.

Technology Stack

The stack is Python with standard data-science and NLP libraries; the Streamlit interface is added only after the core pipeline works.

Purpose	Tool
Language	Python
Core data handling	NumPy, pandas
NLP processing	NLTK, spaCy (where appropriate)
Feature extraction and classification	scikit-learn
Word embeddings	Gensim
Visualization	Matplotlib
Interface	Streamlit (added last)

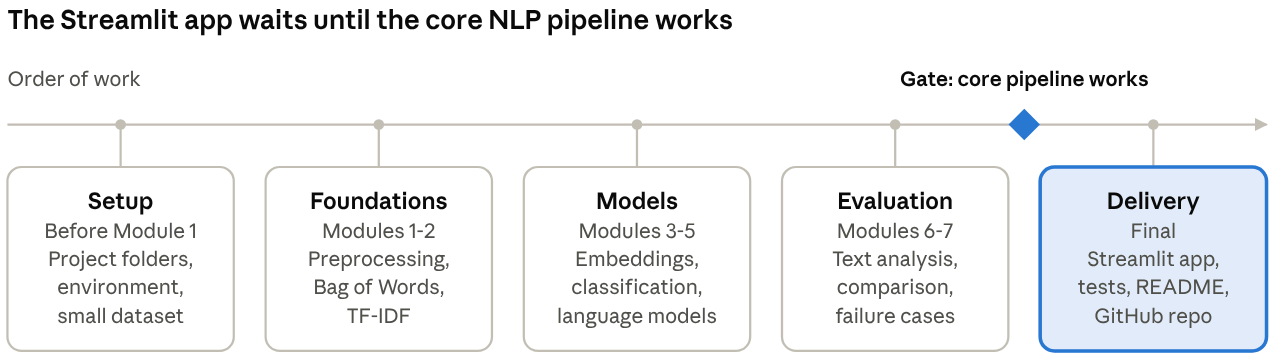
Golden rule: use libraries, but also implement selected algorithms by hand, such as TF-IDF and smoothed bigram probabilities. The manual version teaches the mechanism and lets us verify the library's output, which is exactly what an interviewer will probe.

Folder Structure and Roadmap

Each module gets its own package under `src/` with matching tests, so the pipeline grows one piece at a time.

```
smart-text-intelligence/
├── data/                # raw and processed datasets
├── src/
│   ├── preprocessing/   # Module 1
│   ├── representation/  # Module 2
│   ├── embeddings/      # Module 3
│   ├── classification/  # Module 4
│   ├── language_model/  # Module 5
│   ├── analysis/        # Module 6
│   └── evaluation/      # Module 7
├── tests/               # unit tests per module
├── notebooks/           # experiments and visualizations
├── app/                 # Streamlit UI (built last)
├── requirements.txt
└── README.md
```

The work runs in five phases, shown below. A small, familiar dataset such as SMS Spam is a good first choice because it suits classification and is light enough to iterate on quickly.



roadmap · 5 phases, 1 gate

Phases run in order, and the app is the only piece held back until the pipeline works.

Current Progress

The theory for nearly every module is already studied, and the 20-question NLP assessment scored 14/20 (70%), so the next job is implementation plus targeted revision.

Already studied: NLP fundamentals and pipeline, preprocessing and tokenization, stemming and lemmatization, one-hot encoding and Bag of Words, n-grams and TF-IDF, word embeddings, Word2Vec (CBOW and Skip-gram), GloVe and FastText, statistical n-gram language models, conditional probabilities, and Laplace smoothing.

The assessment flagged six revision areas. Each is practised where it is used in the build, rather than revised in isolation:

Revision area	Practised in
Preprocessing pitfalls	Module 1 and the Module 7 preprocessing study
Bigram probability calculations	Module 5 manual bigram model
Laplace smoothing	Module 5 zero-frequency experiments
CBOW vs Skip-gram	Module 3 Gensim training comparison
Static vs contextual embeddings	Module 3 OOV and ambiguity examples
Choosing the right NLP method for a task	Module 7 BoW vs TF-IDF vs embeddings comparison

Final Deliverables

The project is complete when every item below exists and can be explained without notes.

- [ ] Modular Python codebase, one package per module
- [ ] Working NLP application (Streamlit)
- [ ] Multiple NLP methods implemented and compared (BoW, TF-IDF, embeddings)
- [ ] A suitable dataset with documented evaluation results
- [ ] Unit tests for each module
- [ ] Documented limitations and failure cases
- [ ] README covering architecture, setup, algorithms, and results
- [ ] Portfolio-ready GitHub repository

Two rules keep the scope honest: no RAG, LLM APIs, vector databases, or generative AI features, and no advanced concept is introduced before its prerequisites are covered.

Next Steps

Module 1 (Text Preprocessing) starts with project setup, and each component is explained before it enters the codebase.

1. Create the folder structure and a virtual environment with `requirements.txt`.
2. Choose one small text dataset (SMS Spam is the suggested default).
3. Build the text cleaner: lowercasing, special characters, and punctuation handling.
4. Add sentence splitting and word tokenization.
5. Add stop-word removal, noting where it hurts (for example, removing "not" flips sentiment).
6. Add stemming and lemmatization, and compare their outputs.
7. Handle contractions and negation, then write unit tests for every step.